

mcp-github

Agentic CI/CD Pipeline on Lyzr Studio

Complete Setup Guide — Prompts, API Walkthroughs & Agent Configuration

studio.lyzr.ai

Component	Detail
MCP Server	mcp-github (port 8003, SSE transport)
Platform	studio.lyzr.ai — Lyzr Agent Studio
GitHub API	REST v3 (api.github.com)
Runtime	Python 3.10+ uvicorn ASGI
Tools Exposed	8 tools: branch, commit, PR, CI, coverage, security, merge
Agent Roles	Coder QA Security Verifier

1 Prerequisites & GitHub API Setup

1.1 GitHub Personal Access Token

The MCP server authenticates every call to GitHub using a Bearer token. You must create a Fine-grained Personal Access Token (PAT) with the correct scopes.

Step-by-step: Create a Fine-Grained PAT

1. Go to github.com → click your avatar (top-right) → Settings
2. Scroll down the left sidebar → Developer settings → Personal access tokens → Fine-grained tokens
3. Click Generate new token
4. Token name: mcp-github-lyzr-agent
5. Expiration: 90 days (recommended; set a calendar reminder to rotate)
6. Resource owner: select your org or personal account
7. Repository access: Select repositories → choose your target repo
8. Permissions — expand Repository permissions and grant:

Permission	Access Level
Contents	Read & Write (commit files, read branches)
Pull requests	Read & Write (open, merge PRs)
Checks	Read (get CI results)
Actions	Read & Write (trigger security scan workflow)
Metadata	Read (mandatory baseline)

9. Click Generate token → copy the value immediately (shown only once)

! Store the token in a password manager NOW. GitHub will not show it again after you navigate away.

1.2 Environment Variables

Create a .env file in the same directory as mcp_github.py:

```
# .env
GITHUB_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
GITHUB_OWNER=your-org-or-username
GITHUB_REPO=your-repository-name
```

i GITHUB_OWNER is your GitHub organization slug or personal username — NOT a display name.

1.3 Python Dependencies

Install all required packages into your environment:

```
pip install mcp uvicorn starlette httpx python-dotenv
```

Recommended: use a virtual environment to keep dependencies isolated.

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install mcp uvicorn starlette httpx python-dotenv
```

2 Running the MCP Server

2.1 Start the Server

```
# From the directory containing mcp_github.py
python mcp_github.py

# Expected output:
# mcp-github running on :8003
```

The server exposes two endpoints:

- GET /sse — SSE stream for Lyzr Studio to connect to
- POST /messages/ — message handling endpoint

i For production, run behind a process manager (e.g. systemd, PM2, or Docker) and expose via a reverse proxy with TLS.

2.2 Verify the Server is Reachable

Test the SSE endpoint before registering in Lyzr Studio:

```
curl -N http://localhost:8003/sse
# Should return an SSE stream header (event: message, etc.)
```

If the server runs on a remote host, replace localhost with the server's public IP or hostname. Lyzr Studio must be able to reach this URL.

2.3 Exposing Locally for Testing (ngrok)

If developing locally and Lyzr Studio cannot reach your machine directly, use ngrok to create a tunnel:

```
ngrok http 8003
# Copy the Forwarding URL e.g. https://abc123.ngrok-free.app
# Use: https://abc123.ngrok-free.app/sse in Lyzr Studio
```

3 Registering the MCP Server in Lyzr Studio

3.1 Add the MCP Server

10. Log in to studio.lyzr.ai
11. In the left sidebar, navigate to MCP Servers (or Integrations → MCP)
12. Click Add Server / + New MCP Server
13. Fill in the form:

Field	Value
Name	mcp-github
URL	<code>http://YOUR-HOST:8003/sse</code>
Transport	SSE (Server-Sent Events)
Auth	None (token is handled server-side in <code>.env</code>)
Description	GitHub CI/CD automation — branch, commit, PR, CI, security, merge

14. Click Save / Connect
15. Lyzr Studio will call the `/sse` endpoint and enumerate the 8 tools. You should see them listed in the server detail view.

3.2 Confirm Tool Discovery

After connecting, verify Lyzr Studio has discovered all 8 tools:

Tool Name	Purpose
<code>create_branch</code>	Create a feature branch from main (or any base)
<code>commit_files</code>	Push a single file to a branch
<code>open_pull_request</code>	Open a PR from branch to base
<code>get_pr_status</code>	Check PR state + all CI check statuses
<code>get_ci_result</code>	Detailed CI run info + proof hash for verification
<code>get_coverage_report</code>	Parse test coverage % from PR bot comments
<code>run_security_scan</code>	Trigger GitHub Actions security.yml workflow
<code>merge_pull_request</code>	Squash-merge a PR after verification

4 Agent Design & System Prompts

The pipeline uses four specialized agents. Each agent is scoped to a specific role and only has access to the tools it needs. Define each as a separate agent in Lyzr Studio, attaching the mcp-github server to all four.

Agent 1 — Coder Agent

Tools: create_branch, commit_files, open_pull_request

System Prompt — Coder Agent

```
You are a Coder Agent in an automated CI/CD pipeline powered by GitHub.
Your sole responsibility is to implement tasks by writing code and pushing it
to GitHub via the available tools.

## Workflow (follow this order strictly)
1. Call create_branch with branch name='feature/<task-slug>' and
   from_branch='main' to create an isolated branch.
2. For each file that needs to be created or modified, call commit_files
   with the branch_name, the exact file_path within the repo, the complete
   file_content as a string, and a descriptive commit_message.
3. Once ALL files are committed, call open_pull_request with the branch_name,
   a concise title, a detailed body (what changed and why), and base_branch='main'.
4. Return the prNumber and url from open_pull_request so downstream agents
   can act on this PR.

## Rules
- Never skip step 1. Always create the branch first.
- Commit files one at a time using separate commit_files calls.
- Never open the PR until all commits are done.
- The PR body must include: task description, files changed, and any
  known risks or edge cases.
- Do not merge or close the PR. That is the Verifier Agent's job.
- If create_branch fails with a 422 (branch exists), append a counter
  suffix e.g. 'feature/<task-slug>-2' and retry.
```

Example User Prompt — Coder Agent

```
Implement a rate-limiter middleware for the Express API.
Task slug: rate-limiter-middleware
Files to create:
- src/middleware/rateLimiter.js (the middleware implementation)
- src/middleware/rateLimiter.test.js (unit tests using Jest)
- docs/rate-limiter.md (usage documentation)

Use express-rate-limit v7. Default: 100 requests per 15-minute window.
Configurable via environment variables RATE_LIMIT_MAX and RATE_LIMIT_WINDOW_MS.
```

Agent 2 — QA Agent

Tools: `get_pr_status`, `get_ci_result`, `get_coverage_report`

System Prompt — QA Agent

You are a QA Agent in an automated CI/CD pipeline.
Your job is to verify that a pull request's automated tests have passed
and that code coverage meets the minimum threshold before signalling approval.

Workflow

1. Receive `prNumber` from the Coder Agent.
2. Call `get_pr_status(prNumber)` to check the overall PR state and CI checks.
 - If checks are still running (`status != 'completed'`), wait 60 seconds and retry. Retry up to 10 times before failing.
 - If any check has `conclusion='failure'`, report the failing check names and STOP. Do not proceed.
3. Call `get_ci_result(prNumber)` to get the detailed CI output and proof hash.
4. Call `get_coverage_report(prNumber)` to retrieve the coverage percentage.
 - If `coveragePct` is null, report: 'Coverage data not found. Ensure the CI workflow posts a comment containing Coverage: <N>%'
 - If `coveragePct < 80`, report the shortfall and STOP.
5. If all checks pass and coverage `>= 80%`, output a structured QA Report.

QA Report format (output this exactly)

```
QA_REPORT:
  prNumber: <N>
  allChecksPassed: true
  coveragePct: <N>
  ciOutputHash: <hash>
  verdict: APPROVED
```

Rules

- Never approve a PR with failing checks.
- Never approve a PR with coverage below 80%.
- Do not merge. Pass the QA REPORT to the Security Agent.

Example User Prompt — QA Agent

The Coder Agent has opened PR #47 for the rate-limiter middleware.
Please verify CI and coverage before we proceed to security scanning.

Agent 3 — Security Agent

Tools: `run_security_scan`, `get_ci_result`, `get_pr_status`

System Prompt — Security Agent

You are a Security Agent in an automated CI/CD pipeline.
Your job is to trigger and evaluate the SAST and dependency security scan
on a pull request, then decide whether it is safe to merge.


```

## Pre-condition
Only run if you have received a QA_REPORT with verdict: APPROVED.
Refuse to proceed if the QA verdict is missing or not APPROVED.

## Workflow
1. Call run_security_scan(prNumber).
   The tool triggers .github/workflows/security.yml on the PR's branch.
2. Wait 120 seconds (the scan typically takes 90-180 seconds).
3. Call get_ci_result(prNumber) to check if the security workflow has concluded.
   - If allConcluded is false, wait another 60 seconds and retry (up to 5 times).
4. Call get_pr_status(prNumber) and check the checks list for any check whose
   name contains 'security'. Evaluate its conclusion.
5. If the security check passed (conclusion='success'):
   Output a SECURITY_REPORT with verdict: CLEARED
6. If the security check failed:
   Output a SECURITY_REPORT with verdict: BLOCKED and list findings.

## SECURITY_REPORT format
SECURITY_REPORT:
  prNumber: <N>
  scanTriggered: true
  securityCheckConclusion: success | failure
  verdict: CLEARED | BLOCKED
  notes: <any relevant findings>

## Rules
- Never merge the PR. Only the Verifier Agent merges.
- If verdict is BLOCKED, describe which check failed so the Coder Agent
  can fix the issue.

```

Example User Prompt — Security Agent

```

QA has approved PR #47. QA_REPORT below:
QA REPORT:
  prNumber: 47
  allChecksPassed: true
  coveragePct: 87
  ciOutputHash: a3f9c2e1b0d4e5f16
  verdict: APPROVED

Please trigger the security scan and report results.

```

Agent 4 — Verifier Agent

Tools: merge_pull_request, get_pr_status

System Prompt — Verifier Agent

```

You are a Verifier Agent – the final gatekeeper in an automated CI/CD pipeline.
You merge pull requests ONLY when both QA and Security have given clearance.

## Pre-conditions (ALL must be true before merging)
1. You have a QA_REPORT with verdict: APPROVED
2. You have a SECURITY_REPORT with verdict: CLEARED
3. Both reports reference the same prNumber

```

```

## Workflow
1. Validate both reports are present and both verdicts match the criteria above.
   If either is missing or not approved/cleared, output MERGE_BLOCKED and stop.
2. Call get_pr_status(prNumber) one final time.
   - Confirm state is 'open' and mergeable is true.
   - If not mergeable, report the reason and stop.
3. Call merge_pull_request(prNumber, merge_method='squash').
4. Confirm merged is true in the response.
5. Output a MERGE_RECEIPT.

## MERGE_RECEIPT format
MERGE_RECEIPT:
  prNumber: <N>
  merged: true
  mergeSha: <sha>
  qaVerdict: APPROVED
  securityVerdict: CLEARED
  mergeMethod: squash
  timestamp: <ISO-8601 timestamp>

## Rules
- NEVER merge if QA verdict is not APPROVED.
- NEVER merge if Security verdict is not CLEARED.
- NEVER merge if get_pr_status returns mergeable: false.
- If merge fails, do not retry automatically – report the error for human review.

```

Example User Prompt — Verifier Agent

All checks are complete for PR #47. Reports below:

```

QA_REPORT:
  prNumber: 47
  allChecksPassed: true
  coveragePct: 87
  ciOutputHash: a3f9c2e1b0d4e5f16
  verdict: APPROVED

SECURITY_REPORT:
  prNumber: 47
  scanTriggered: true
  securityCheckConclusion: success
  verdict: CLEARED
  notes: No vulnerabilities found in SAST or dependency scan.

Please verify and merge.

```

5 Pipeline Orchestration in Lyzr Studio

5.1 Create a Multi-Agent Pipeline

In Lyzr Studio, create a Pipeline (or Workflow) that chains the four agents in sequence. Each agent's output becomes the next agent's input.

Step	Agent → Action
1	User submits task description → Coder Agent
2	Coder outputs prNumber → QA Agent
3	QA outputs QA_REPORT → Security Agent
4	Security outputs SECURITY_REPORT → Verifier Agent
5	Verifier merges and outputs MERGE_RECEIPT → User

5.2 Tool Access Scoping

Restrict each agent to only the tools it needs. In Lyzr Studio, when configuring each agent, use the tool allowlist:

Agent	Allowed Tools
Coder	create_branch, commit_files, open_pull_request
QA	get_pr_status, get_ci_result, get_coverage_report
Security	run_security_scan, get_ci_result, get_pr_status
Verifier	merge_pull_request, get_pr_status

5.3 CI Workflow Requirement for Coverage

The `get_coverage_report` tool reads coverage data from PR bot comments. Your CI workflow must post a comment in the format below after running tests:

```
# Example GitHub Actions step to post coverage
- name: Comment coverage
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue number: context.issue.number,
```

```
owner: context.repo.owner,  
repo: context.repo.repo,  
body: `Coverage: ${process.env.COVERAGE_PCT}%`  
})
```

! The comment must contain the string 'Coverage:' followed by a percentage (e.g. Coverage: 87%). The tool parses this pattern specifically.

5.4 Security Workflow Requirement

The `run_security_scan` tool dispatches the workflow at `.github/workflows/security.yml`. Create this file in your repo:

```
# .github/workflows/security.yml  
name: Security Scan  
on:  
  workflow_dispatch:  
    inputs:  
      pr_number:  
        description: PR number  
        required: true  
jobs:  
  sast:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Run Semgrep SAST  
        uses: semgrep/semgrep-action@v1  
        with:  
          config: p/default  
  dependency-scan:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Run npm audit  
        run: npm audit --audit-level=high
```

Error / Symptom	Resolution
401 Unauthorized from GitHub API	Token expired or wrong scope. Regenerate PAT with correct permissions (Section 1.1).
422 Unprocessable Entity on create_branch	Branch already exists. Append -2 to the branch name and retry.
get_coverage_report returns found: false	CI is not posting a comment with 'Coverage: N%'. Add the Actions step from Section 5.3.
run_security_scan returns 404	The file .github/workflows/security.yml does not exist in the repo. Create it (Section 5.4).
Lyzzr Studio cannot connect to /sse	Check firewall rules. Use ngrok for local dev (Section 2.3). Confirm server is running on port 8003.
merge_pull_request returns mergeable: false	There are merge conflicts. The Coder Agent must rebase the branch against main.
SSE connection drops after a few minutes	Add uvicorn --timeout-keep-alive 120 flag, or configure your reverse proxy to keep SSE connections alive.

Pipeline Flow Summary

Coder → QA → Security → Verifier → Merged to main